

1.

Comparisons with NULL do not give TRUE or FALSE but UNKNOWN, so missing values cannot be directly compared.

In aggregate calculations, NULL values are ignored except when counting rows in total. When grouping, all missing values are treated as equal and form one common group.

2.

1. Selection (σ)

a. Purpose: Extracts rows (tuples) from a relation based on a condition.

b. Example: From all Books, select those where condition = 'New'.

2. Projection (π)

a. Purpose: Extracts specific columns (attributes) from a relation.

b. Example: From Books, keep only title and author.

3. Cartesian Product ($\times$)

a. Purpose: Combines every tuple of one relation with every tuple of another relation.

b. Example: Combine all Books with all Publishers, producing every possible pair.

4. Union ($\cup$)

a. Purpose: Returns all tuples from two relations, removing duplicates.

b. Example: A set of oldBooks $\cup$ newBooks gives all books.

5. Set Difference ($-$)

a. Purpose: Returns tuples in one relation but not in the other.

b. Example: allBooks $-$ borrowedBooks gives books that are not borrowed.

3.

Selection (σ) : chooses rows based on a condition.

Projection (π) : chooses specific columns.

Cartesian Product ($\times$) : pairs every row of one relation with every row of another.

Union ($\cup$) : combines tuples from two relations.

Set Difference ($-$) : finds tuples in one relation but not in another.

Join ($\bowtie$) : Cartesian Product ($\times$) + Selection (σ)

A join is a Cartesian Product followed by a Selection.

Example: Books (with publisher ID) $\bowtie$ Publishers (with publisher ID) = list of books with their publisher names.

Intersection ($\cap$) : $R \cap S = R - (R - S)$

Intersection is built using Set Difference.

Students in football $\cap$ students in basketball = students who are in both teams.

Division = Projection + Product + Difference

Division uses Projection, Cartesian Product, and Difference.

Division is used to find entities that are related to all values of another set. It is defined using projection, Cartesian product, and difference.

If we have a list of students and the courses they have taken, and a separate list of all core courses that must be completed.

4.

1. Theta Join ($\bowtie_{\theta}$)

A join that allows a general condition (θ), not just equality.

Example:

Books $\bowtie_{(\text{Book.cost} < \text{Publisher.avgPrice})}$ Publishers

This gives all books whose cost is less than the publisher's average price.

The condition can be $<$, $>$, $=$, $\leq$, $\geq$, $\neq$.

2. Equijoin ($\bowtie_{=}$)

A special case of theta join where the condition is only equality.

Example : Books $\bowtie_{(\text{Book.publisherID} = \text{Publisher.publisherID})}$ Publishers

This gives all books along with publisher details, but both relations still keep the duplicate publisherID columns.

Equality only, may have duplicate join columns.

3. Natural Join ($\bowtie$)

An equijoin without duplicates in the common attribute(s). It automatically matches attributes with the same name.

Example :

Books $\bowtie$ Publishers (on publisherID)

The result shows each book with its publisher's info, but publisherID appears only once.

Cleaner than equijoin, avoids repeating columns.

4. Outer Join ($\bowtie\leftarrow$, $\rightarrow\bowtie$, $\bowtie\leftarrow\rightarrow$)

Extends natural join to keep tuples that don't have matches, filling missing values with NULLs.

Types of outer join:

- a. Left Outer Join ($\bowtie\leftarrow$): Keeps all rows from left table.
- b. Right Outer Join ($\rightarrow\bowtie$): Keeps all rows from right table.
- c. Full Outer Join ($\bowtie\leftarrow\rightarrow$): Keeps all rows from both tables.

Example :

Left outer join between Books and Publishers $\rightarrow$ Books with publisher info, plus books that don't have a publisher recorded (shown with NULL publisher fields).

5. Semijoin ($\ltimes$ or $\ltimes$)

Semijoin is like a join, but only keeps attributes from one side of the join.

Example :

Books $\ltimes$ Publishers $\rightarrow$ List of books that have a matching publisher, but without publisher attributes.

Useful for filtering one relation based on matches in another, but not combining attributes.

4.

Theta Join ($\bowtie\theta$): Join with a general condition (any comparison operator).

For eg:

Relation Student(sid, name, age, major)

Relation Course(cid, cname, credits)

Condition: Student.age > Course.credits

Result: Pairs of students and courses where the student's age is greater than the number of course credits.

Equijoin ($\bowtie$): A special case of theta join where the condition is only equality (=).

For eg:

Relation Student(sid, name, majorID)

Relation Department(majorID, deptName)

Condition: Student.majorID = Department.majorID

Result: Students matched with their corresponding department information.

5.

(a) $\Pi_{\text{hotelNo}} (\sigma_{\text{price} > 50} (\text{Room}))$

Returns the set of hotel numbers for which there exists a room whose price is greater than 50. (We keep only the hotelNo values from the Room tuples with price > 50; duplicates are removed by projection.)

(b) $\sigma_{\{\text{Hotel.hotelNo} = \text{Room.hotelNo}\}} (\text{Hotel} \times \text{Room})$

Returns all combined Hotel–Room pairs for which the hotel number is the same in both relations — i.e., every room paired with its corresponding hotel. (This is an inner join on hotelNo producing tuples that include attributes from both Hotel and Room.)

(c) $\Pi_{\text{hotelName}} (\text{Hotel} \bowtie_{\{\text{Hotel.hotelNo} = \text{Room.hotelNo}\}} (\sigma_{\text{price} > 50} (\text{Room})))$

First keep only rooms priced > 50, then join those rooms to their hotels, and finally return the hotel names. The result is the set of hotel names for hotels that have at least one room priced over 50.

(d) $\text{Guest} \bowtie (\sigma_{\text{dateTo} \geq \text{'1-Jan-...'}} (\text{Booking}))$

Keeps only bookings that end on or after the given date, then pairs those bookings with the corresponding guest records. The result is guest–booking pairs (guest information together with only those bookings whose dateTo is $\geq$ the specified date).

(e) $\text{Hotel} \triangleright_{\{\text{Hotel.hotelNo} = \text{Room.hotelNo}\}} (\sigma_{\text{price} > 50} (\text{Room}))$

(semijoin) Returns the Hotel tuples for hotels that have at least one room with price > 50. In other words, it filters Hotel to those hotels that have a matching Room meeting price > 50 (only hotel attributes are kept).

(f) $\Pi_{\text{guestName, hotelNo}} (\text{Booking} \bowtie_{\{\text{Booking.guestNo} = \text{Guest.guestNo}\}} \text{Guest}) \div \Pi_{\text{hotelNo}}(\sigma_{\text{city} = \text{'London'}}(\text{Hotel}))$

First produce pairs of (guestName, hotelNo) for every booking (by joining Booking with Guest). Then divide by the set of hotel numbers of hotels in London. The result is the set of guest names of those guests who have booked every hotel in London — i.e., guests that have stayed at all London hotels.

6.

1. Select relevant departments (IT and Planning):

DeptSel : $\sigma_{\text{deptName} = \text{'IT'} \vee \text{deptName} = \text{'Planning'}}(\text{Department})$
 $\text{DeptSel} \leftarrow \sigma_{\text{deptName} = \text{'IT'} \vee \text{deptName} = \text{'Planning'}}(\text{Department})$

2. Join employees with selected departments:

EmpDept : $\text{Employee} \bowtie_{\text{Employee.deptNo} = \text{DeptSel.deptNo}} \text{DeptSel}$
 $\text{EmpDept} \leftarrow \text{Employee} \bowtie_{\text{Employee.deptNo} = \text{DeptSel.deptNo}} \text{DeptSel}$

3. Select employees born after 1990:

EmpAfter1990 : $\sigma_{\text{DOB} > \text{'1990-12-31'}}(\text{EmpDept})$
 $\text{EmpAfter1990} \leftarrow \sigma_{\text{DOB} > \text{'1990-12-31'}}(\text{EmpDept})$

4. Project the required report attributes:

Result : $\pi_{\text{empNo, fName, lName, DOB, sex, position, deptName}}(\text{EmpAfter1990})$
 $\text{Result} \leftarrow \pi_{\text{empNo, fName, lName, DOB, sex, position, deptName}}(\text{EmpAfter1990})$

Final Expression:

$\text{Result} = \pi_{\text{empNo, fName, lName, DOB, sex, position, deptName}}(\sigma_{\text{DOB} > \text{'1990-12-31'}}((\text{Employee} \bowtie_{\text{Employee.deptNo} = \text{Department.deptNo}} (\sigma_{\text{deptName} = \text{'IT'} \vee \text{deptName} = \text{'Planning'}}(\text{Department}))))))$
 $\text{Result} = \pi_{\text{empNo, fName, lName, DOB, sex, position, deptName}}(\sigma_{\text{DOB} > \text{'1990-12-31'}}((\text{Employee} \bowtie_{\text{Employee.deptNo} = \text{Department.deptNo}} (\sigma_{\text{deptName} = \text{'IT'} \vee \text{deptName} = \text{'Planning'}}(\text{Department}))))))$

6.

a) List all employees

- Relational Algebra: $\pi \text{ fName, lName } (\text{Employee})$
- Tuple Relational Calculus: $\{ e.\text{fName}, e.\text{lName} \mid \text{Employee}(e) \}$
- Domain Relational Calculus: $\{ \langle F, L \rangle \mid \exists A, D, S, P, N (\text{Employee}(F, L, A, D, S, P, N)) \}$

b) List all the details of employees who are female and born after 1990

- Relational Algebra: $\sigma \text{ sex}='F' \wedge \text{DOB} > '1990-12-31' (\text{Employee})$
- Tuple Relational Calculus: $\{ e \mid \text{Employee}(e) \wedge e.\text{sex}='F' \wedge e.\text{DOB} > '1990-12-31' \}$
- Domain Relational Calculus: $\{ \langle E, F, L, A, D, S, P, N \rangle \mid \text{Employee}(E, F, L, A, D, S, P, N) \wedge S='F' \wedge D > '1990-12-31' \}$

c) List all employees who are not managers and are paid more than \$1500

- Relational Algebra: $\sigma \text{ position} \neq 'Manager' \wedge \text{salary} > 1500 (\text{Employee})$
- Tuple Relational Calculus: $\{ e \mid \text{Employee}(e) \wedge e.\text{position} \neq 'Manager' \wedge e.\text{salary} > 1500 \}$
- Domain Relational Calculus: $\{ \langle E, F, L, A, D, S, P, N \rangle \mid \text{Employee}(E, F, L, A, D, S, P, N) \wedge P \neq 'Manager' \wedge e.\text{salary} > 1500 \}$

d) Produce a list of the names and addresses of all employees who work for the IT department

- Relational Algebra: $\pi \text{ fName, lName, address } (\sigma \text{ deptName}='IT' (\text{Employee} \bowtie \text{Department}))$
- Tuple Relational Calculus: $\{ \langle e.\text{fName}, e.\text{lName}, e.\text{address} \rangle \mid \text{Employee}(e) \wedge \text{Department}(d) \wedge e.\text{deptNo}=d.\text{deptNo} \wedge d.\text{deptName}='IT' \}$
- Domain Relational Calculus: $\{ \langle F, L, A \rangle \mid \exists Dn (\text{Employee}(E, F, L, A, DOB, S, P, N) \wedge \text{Department}(N, Dn, M) \wedge Dn='IT') \}$

e) Produce a list of the names of all employees who work on the SCCS project

- Relational Algebra: π fName, lName (Employee $\bowtie$ WorksOn $\bowtie$ σ projName='SCCS' (Project))
- Tuple Relational Calculus: $\{ \langle e.fName, e.lName \rangle \mid \text{Employee}(e) \wedge \text{WorksOn}(w) \wedge \text{Project}(p) \wedge e.empNo=w.empNo \wedge w.projNo=p.projNo \wedge p.projName='SCCS' \}$
- Domain Relational Calculus: $\{ \langle F, L \rangle \mid \exists Pn, Pr (\text{Employee}(E, F, L, A, DOB, S, P, N) \wedge \text{WorksOn}(E, Pr, Dw, Hw) \wedge \text{Project}(Pr, Pn, N) \wedge Pn='SCCS') \}$

f) Produce a complete list of all managers who are due to retire this year, in alphabetical order of surname

- Relational Algebra: τ lName (σ position='Manager' $\wedge$ retirementYear=2025 (Employee))
- Tuple Relational Calculus: $\{ e \mid \text{Employee}(e) \wedge e.position='Manager' \wedge e.retirementYear=2025 \}$ ordered by e.lName
- Domain Relational Calculus: $\{ \langle E, F, L, A, D, S, P, N \rangle \mid \text{Employee}(E, F, L, A, D, S, P, N) \wedge P='Manager' \wedge e.retirementYear=2025 \}$

g) Find out how many managers are female

- Relational Algebra: COUNT (σ position='Manager' $\wedge$ sex='F' (Employee))
- Tuple Relational Calculus: $\{ \text{count}(e) \mid \text{Employee}(e) \wedge e.position='Manager' \wedge e.sex='F' \}$
- Domain Relational Calculus: $\{ \text{count}(E) \mid \text{Employee}(E, F, L, A, DOB, S, P, N) \wedge P='Manager' \wedge S='F' \}$

h) Produce a report of all projects under the IT department

- Relational Algebra: π projNo, projName (Project $\bowtie$ σ deptName='IT' (Department))
- Tuple Relational Calculus: $\{ \langle p.projNo, p.projName \rangle \mid \text{Project}(p) \wedge \text{Department}(d) \wedge p.deptNo=d.deptNo \wedge d.deptName='IT' \}$
- Domain Relational Calculus: $\{ \langle Pr, Pn \rangle \mid \exists N (\text{Project}(Pr, Pn, N) \wedge \text{Department}(N, 'IT', M)) \}$

i) Using the union operator, retrieve the list of employees who are neither managers nor supervisors. Attributes to be retrieved are first name, last name, position, sex, and department number

- Relational Algebra: $\pi \text{ fName, lName, position, sex, deptNo } (\sigma \text{ position} \neq \text{'Manager'} \wedge \text{position} \neq \text{'Supervisor'} (\text{Employee}))$
- Tuple Relational Calculus: $\{ \langle e.\text{fName}, e.\text{lName}, e.\text{position}, e.\text{sex}, e.\text{deptNo} \rangle \mid \text{Employee}(e) \wedge e.\text{position} \neq \text{'Manager'} \wedge e.\text{position} \neq \text{'Supervisor'} \}$
- Domain Relational Calculus: $\{ \langle F, L, P, S, N \rangle \mid \text{Employee}(E, F, L, A, \text{DOB}, S, P, N) \wedge P \neq \text{'Manager'} \wedge P \neq \text{'Supervisor'} \}$

7.

Person (pId, pName, pPhone, pAddress)

pId is the primary key

Owner (ownerId)

ownerId is a foreign key from Person table

Stable (sId, sName, sPhone)

sId is the primary key

Horse (regNum, name, gender, type, dateBought, purchasePrice, sireRegNum->FK, damRegNum->FK)

regNum is the primary key

sireRegNum and **damRegNum** are foreign keys from Horse table (self-relationship)

OwnedBy (regNum->FK, ownerId->FK, percentage)

regNum is a foreign key from Horse table

ownerId is a foreign key from Owner table

Trainer (trainerId->FK, salary)

trainerId is a foreign key from Person table

Jockey (jockeyId->FK, weight, weightDate)

jockeyId is a foreign key from Person table

TrainerJockey (trainerId->FK, jockeyId)

trainerId is a foreign key from Trainer table

jockeyId is a foreign key from Jockey table

WorksFor (trainerId->FK, sId->FK, contact)
trainerId is a foreign key from Trainer table
sId is a foreign key from Stable table

TrainedBy (regNum->FK, trainerId->FK)
regNum is a foreign key from Horse table
trainerId is a foreign key from Trainer table

Race (racId, purse)
racId is the primary key

RaceSchedule (scheduleId, sDate)
scheduleId is the primary key

HasRace (scheduleId->FK, racId->FK)
scheduleId is a foreign key from RaceSchedule table
racId is a foreign key from Race table

Track (tId, tName)
tId is the primary key

HasSchedule (tId->FK, scheduleId->FK)
tId is a foreign key from Track table
scheduleId is a foreign key from RaceSchedule table

Entry (racId->FK, regNum->FK, rNumber, gate, finalPos)
racId is a foreign key from Race table
regNum is a foreign key from Horse table

JRaces ((racId, regNum) -> FK, jockeyId->FK)
racId, regNum is a foreign key from Entry table
jockeyId is a foreign key from Jockey table